Python Fundamentals · Exercises

Part 2 · Decisions and Data · Sheet 2 · Strings in Depth

> **How to work.** Create a folder `part2-strings`, open it in VS Code and create a virtual environment in it. Write **one script per exercise**, named after it: `ex_1_01.py`, `ex_2_04.py`… and run each one from the terminal. **No loops yet**: everything here is solved with what the course has shown so far. When a test table is given, run your script with every row.
>
> 1. **Easy**: one notion at a time, as seen in the course.
> 2. **Intermediate**: several notions together.
> 3. **Hard**: small problems; think before you type, and check every test case.
> 4. **Very Hard**: mini-projects, guided step by step.

Level 1 · Easy **Level 2 · Intermediate** **Level 3 · Hard** **Level 4 · Very Hard**

Level 1 · Easy

1.1 First and Last

Course, section 02

Ask the user for a word. Display its first character, its last character (with a negative index) and its number of characters.

Input	Expected output

`Python`	`First: P`
	`Last: n`
	`Length: 6`

1.2 Slices

Course, section 02

With `word = "Programming"`, display with one slice each: `Pro`, `gram`, `ming`, the last 3 letters with a negative start, and the word without its first and last letters. Draw the index ruler on paper first.

1.3 Slicing with a Step

Course, section 02

With `digits = "0123456789"`, display with one slice each, using a **step** (`s[start:stop:step]`):

1. the even digits, `02468`, then the odd digits, `13579`;
2. one digit out of three, `0369`;
3. the digits from 2 to 8 included, two by two, `2468`;
4. the whole string backwards, `9876543210`;
5. the even digits backwards, `86420`;
6. the digits from 7 down to 3, `76543`.

Finally, display `digits[2:7:-1]` and explain the result in a comment.

© 2026 PALISSON Antoine · CC BY-NC-ND 4.0

1.4 Clean Up

Course, section 02

With `text = "  hELLO  wORLD  "`, display the text without the surrounding spaces, then in uppercase, in lowercase, and with each word capitalized (without the surrounding spaces). Finally display `text` itself: has it changed?

1.5 Search and Replace

Course, section 02

With `sentence = "Python is easy to learn and Python is fun"`, display: the index of the first `Python`; the result of searching `Java`; how many times `Python` appears; the sentence with `fun` replaced by `powerful`; whether it starts with `Python`; whether it ends with `!`.

1.6 Identity Card

Course, section 02

Store a name, an age and a city in three variables. With f-strings only (no `+`, no `str()`), display `Ada is 36 and lives in London.` and `Next year, Ada will be 37.`, computed from the variables.

1.7 Number Formats

Course, section 02

With `price = 1234.5678` and `rate = 0.0825`, display with f-strings: the price with 2 decimals; with a thousands separator and 2 decimals; the rate as a percentage with 1 decimal; the price with 2 decimals right-aligned in 12 characters, between square brackets to see the width.

1.8 Read the Error

Course, section 02

Each block below stops with an error. Run them one by one, copy the last line of each message as a comment, then fix the block. For c), use an f-string.

```
# a)
word = "Python"
word[0] = "J"
print(word)

# b)
print(word[6])

# c)
age = 20
print("Age: " + age)
```

Level 2 · Intermediate

2.1 Initials

Course, section 02

Ask for a first name and a last name, typed in any case, and display the initials in uppercase, each followed by a dot.

Input	Expected output
`ada, lovelace`	`A. L.`
`Grace, HOPPER`	`G. H.`

© 2026 PALISSON Antoine · CC BY-NC-ND 4.0

2.2 Email Check

Course, section 02

Ask for an email address. It is accepted if it contains `@`, contains no space, and ends with `.com` or `.fr`. Otherwise display the **first** reason why it is refused, in this order: missing `@`, space, domain.

Input	Expected output
`ada@mail.com`	`Accepted`
`ada.mail.com`	`Refused: no @`
`ada @mail.fr`	`Refused: contains a space`
`ada@mail.org`	`Refused: must end with .com or .fr`

2.3 Yes or No

Course, section 02

Ask `Continue? ` and accept the answer whatever its case and its surrounding spaces: `yes` or `y` display `Confirmed`, `no` or `n` display `Cancelled`, anything else `Please answer yes or no`.

Input	Expected output
`  YES`	`Confirmed`
`n`	`Cancelled`
`maybe`	`Please answer yes or no`

2.4 Username

Course, section 02

Build a username from a first name, a last name and a year of birth, all typed by the user: the first 3 letters of the last name, the first letter of the first name and the last 2 digits of the year, all in lowercase.

Input	Expected output
`Ada, Lovelace, 1815`	`Username: lova15`

2.5 Is It a Whole Number?

Course, section 02

Ask for a value. If it is made of digits only, display it doubled. If it starts with `-` followed by digits only, display `Negative number:` and the number. Otherwise display `Not a whole number`, without ever stopping on a ValueError.

Input	Expected output
`21`	`Doubled: 42`
`-7`	`Negative number: -7`
`3.5`	`Not a whole number`

2.6 Aligned Receipt

Course, section 02

Three items and their prices are stored in six variables: Tea 3.5, Chocolate cake 12.25, Water 1.8. Display a receipt where the names are left-aligned in 16 characters and the prices right-aligned in 8 characters with 2 decimals, then a line of 24 dashes and the total, aligned like the prices.

© 2026 PALISSON Antoine · CC BY-NC-ND 4.0

2.7 Escape Practice

Course, section 02

Display exactly the three lines below, one `print()` each. For the path, use a raw string; for the quotes, choose your quotes well; for the last line, separate the columns with `\t`.

```
C:\Users\ada\new_folder
He said "Python" is not hard
Name:	Ada	Age:	36
```

2.8 Debug with f"{x=}"

Course, section 02

This script should display an average of 15.0, but it does not. Add `print(f"{...=}")` lines to display the intermediate values, find the mistake, fix it, then remove the debug lines.

```
a, b, c = 12, 15, 18
average = a + b + c / 3
print(average)
```

Level 3 · Hard

3.1 Palindrome

Course, section 02

A palindrome reads the same in both directions, ignoring case and spaces: *Kayak*, *Never odd or even*. Ask for a phrase and display whether it is a palindrome. No loop is needed: clean the phrase, then compare it with its reverse.

Input	Expected output
Kayak	Palindrome
Never odd or even	Palindrome
Python	Not a palindrome

3.2 Masked Card Number

Course, section 02

Ask for a card number written in 4 groups of 4 digits separated by spaces. If it has exactly that shape (19 characters, and 16 digits once the spaces are removed), display it masked, only the last group visible: `**** **** **** 3456`. Otherwise display `Invalid card number`.

Input	Expected output
1234 5678 9012 3456	**** **** **** 3456
1234567890123456	Invalid card number
1234 5678 9012 34a6	Invalid card number

© 2026 PALISSON Antoine · CC BY-NC-ND 4.0

3.3 Name Fixer

Course, section 02

Users type their full name carelessly: `" aDA LOVELACE "`. Display it cleaned: no spaces around, one space between the words, each word capitalized. *Hint*: `split()` without argument cuts at any run of spaces; `join()` glues the pieces back.

Input	Expected output
`   aDA    LOVELACE`	`[Ada Lovelace]`
`grace  brewster   HOPPER`	`[Grace Brewster Hopper]`

3.4 Clock Format

Course, section 02

Ask for a time written `h:m`, such as `7:5`. Find the colon with `find()`, cut the hours and the minutes with slices, convert them, and display the time on 2 digits each: `07:05`. Display `Invalid time` when there is no colon, or when the hours are not 0 to 23 or the minutes 0 to 59.

Input	Expected output
`7:5`	`07:05`
`23:59`	`23:59`
`24:00`	`Invalid time`
`715`	`Invalid time`

3.5 Text Statistics

Course, section 02

Ask for a sentence of several words and display: its number of characters, its number of characters without spaces, its number of words, how many times the letter `e` appears (in any case), its first word and its last word. `rfind()` works like `find()`, from the right.

Input	Expected output

`Every exercise needs care`	`Characters: 25`
	`Without spaces: 22`
	`Words: 4`
	`Letter e: 8`
	`First word: Every`
	`Last word: care`

© 2026 PALISSON Antoine · CC BY-NC-ND 4.0

Level 4 · Very Hard

4.1 License Plate Checker

Course, section 02

A French license plate looks like `AB-123-CD`: two letters, a dash, three digits, a dash, two letters. The letters I, O and U are never used. Ask for a plate and say whether it is valid; if not, say why.

1. Clean the plate: no surrounding spaces, uppercase.
2. Check its length (9), then the two dashes, at indices 2 and 6.
3. Cut the three parts with slices; check that the letters are letters and the digits are digits.
4. Check that none of I, O, U appears in the letters.
5. Display `Valid plate:` and the cleaned plate, or the first error found, in this order: `Wrong length`, `Dashes misplaced`, `Wrong characters`, `Forbidden letter`.

Input	Expected output
`ab-123-cd`	`Valid plate: AB-123-CD`
`AB-1234-CD`	`Wrong length`
`AB 123 CD`	`Dashes misplaced`
`AB-12C-CD`	`Wrong characters`
`AO-123-CD`	`Forbidden letter`

4.2 Password Strength

Course, section 02

Ask for a password and give it a score from 0 to 5, one point per rule met: at least 8 characters; at least 12 characters; an uppercase letter; a lowercase letter; a digit. Display the score and a verdict: 0 to 2 `weak`, 3 or 4 `medium`, 5 `strong`.

1. The length rules are two simple comparisons.
2. A password contains an uppercase letter when `password.lower()` is different from `password`. Find the same trick for lowercase letters.
3. For the digits, no loop: remove the ten digits with chained `replace()` calls, then compare the lengths.
4. Add the points with augmented assignment, then display the verdict with `if` / `elif` / `else`.

Input	Expected output
`abc`	`Score: 1/5, weak`
`Password1`	`Score: 4/5, medium`
`Correct-Horse-7`	`Score: 5/5, strong`

© 2026 PALISSON Antoine · CC BY-NC-ND 4.0